Proposed changes for [A92], how a trained policy is chosen and tested, and the lock rule of [A91]

13 November 2026, version 3 (validation on 5 seeds, runtime on the CPU decided)

How to read this

Part 1 says what was decided on 13 November and what was built, in plain words. Part 2 shows the text in the Experiments chapter that would change. Text in green is proposed, ~~struck~~ text would go, black text is already there, and italic text is a mockup instruction. Part 3 lists the questions.

Part 1. What was decided

Validation goes with training, the test is kept apart

This is the usual practice in deep learning. Training saves a checkpoint every three iterations. Each checkpoint plays the validation warehouses, which no run trains on, and the checkpoint to use is chosen there by the rule of [A60]: of the checkpoints that keep up on every validation warehouse, the one with the shortest mean pick service time, or the highest throughput if none keeps up. Only the chosen checkpoint then plays the test warehouses, and only those numbers are reported.

New runs validate every checkpoint while they train, on 5 validation warehouses. The runs already trained validate afterwards, every checkpoint on the same 5 validation warehouses (their settings held 2, and you chose to validate them again on 5), at their training length of 2000 steps. The numbers are the same either way, only the timing differs.

The test

20 test warehouses, up from 6, because 2 of the first 6 (and 8 of 20 at 32 agents) have the layout that can lock. Each test episode is 10000 steps, and the first 5000 are left out as warm-up, as in the Token Passing runs. The policy runs on the GPU and the warehouse simulation on the CPU. The runtime is measured apart on the CPU, three test warehouses per run, one run at a time, so the controllers and Token Passing are timed alike.

The two setups ([A84])

The main setup plays one policy, trained under fixed storage, under fixed and under demand storage, on the same test warehouses. The gain is then storage alone. The second setup plays

each rule with the policy trained under it, so its gain mixes storage and training. Congestion-aware storage waits, because you found an issue in it.

The lock rule

Each test episode now stores the longest time two agents both held their task at the same step. An agent with no task never counts, because an idle agent parks on an endpoint, which hangs off the aisle and blocks no one. The threshold for a lock is set from how these numbers spread once the results are in, so nothing has to be played again. In the traces so far a normal task took 60 to 100 steps, while healthy seeds of barely trained policies reached 400 to 460 steps, so the earlier 500 steps was close to normal waits.

Where it is recorded

Every evaluation writes its rows and an `evaluation.json` beside the runs, and `collect_evaluation.py` adds a `README.md` with the commands, which policy was tested and why, and the fleet's comparison table for both setups. The small files go to `results/L_results/`.

Part 2. The text

2.1 Experiments, "Training"

A controller is used for evaluation only once its training return has levelled off, judged by *criterion*. ~~*Which checkpoint is evaluated, chosen on validation seeds kept apart from the evaluation seeds.*~~ Training saves a checkpoint every three iterations, and every checkpoint plays the validation seeds, warehouses no run trained on. The checkpoint that is tested is chosen there. Of the checkpoints that keep up on every validation seed, it is the one with the shortest mean pick service time. If none keeps up, it is the one with the highest throughput. Only the chosen checkpoint plays the test seeds, so a result is never measured on the warehouses that chose the policy.

2.2 Experiments, "Scoring a condition"

~~*Warm-up. A run starts with no tasks, and the storage rule needs data before its first useful update, so the first timesteps are not typical. State how many are discarded before scoring, and that T covers many storage epochs.*~~ A test run starts with no tasks, and the storage rule needs orders before its first useful update, so the first steps are not typical. Every test run is 10000 steps long and the first 5000 are left out, as in the Token Passing runs. Each condition plays 20 test seeds.

2.3 Experiments, Results, "Keeping up"

Give the number of test seeds that lock, per controller and storage rule. A seed locks when at least two agents each hold the same task for ~~*500 steps or more at the same time, a quarter of the episode*~~ *longer than a threshold at the same time. Each run stores the longest such time. Give the threshold, chosen from how these times spread, and why. A locked seed counts as not keeping up.*

2.4 Experiments, “Compute environment”

> The test results are played with the learned policies on the GPU, which only makes them faster. The runtime κ_T is measured apart, on the CPU as for Token Passing, with each controller alone on the machine on three test seeds of full length.

Part 3. Questions

1. **The lock threshold** is chosen once the fleet 16 results are in. Is that the order you want?
2. **Congestion-aware storage** is paused, both its evaluation and its training. What is the issue you found, so it can be fixed before it runs again?